Exercice

Soit la classe *point* :

```
class point
{
   float x, y ;
  public :
   point (float, float) ;
   void deplace (float, float) ;
   void affiche () ;
}
```

Adapter cette classse, de manière à ce que la fonction membre *affiche* fournisse, en plus des coordonnées du point, le nombre d'objets de type *point*.

solution

```
        /*       fichier POINT5.H         */
        /* déclaration de la classe point */
class point
{
   static nb_pts ;                  // compteur du nombre d'objets créés
   float x, y ;                     // coordonnées (cartésiennes) du point
 public :
   point (float, float) ;           // constructeur
   ~point () ;                      // destructeur
   void deplace (float, float) ;    // déplacement
   void affiche () ;                // affichage
} ;
```

Solution

```
#include "point5.h"
#include <iostream.h>
point::point (float abs, float ord)        // constructeur
{  x = abs ; y = ord ;
   nb_pts++ ;                              //    actualisation nb points
}
point::~point ()                           // destructeur
{  nb_pts-- ;                              //    actualisation nb points
}
void point::deplace (float dx, float dy)
{  x = x + dx ; y = + dy ;
}
void point::affiche ()
{  cout << "Je suis un point parmi " << nb_pts
    << " de coordonnées "<< x << " " << y << "\n" ;
}
```

Exercice

Réaliser une classe *vecteur3d* permettant de manipuler des vecteurs à 3 composantes (de type *float*). On y prévoira :

- un constructeur, avec des valeurs par défaut (0),
- une fonction d'affichage des 3 composantes du vecteur, sous la forme :

```
< composante1, composante2, composante3 >
```

- une fonction permettant d'obtenir la somme de 2 vecteurs,
- une fonction permettant d'obtenir le produit scalaire de 2 vecteurs.

Solution

```
      /* déclaration de la classe vecteur3d */
class vecteur3d
{   float x, y, z ;
  public :
    vecteur3d (float c1=0, float c2=0, float c3=0)  // constructeur
      {  x=c1 ; y=c2 ; z=c3;
      }
    vecteur3d somme (vecteur3d &) ;       // somme (résultat par valeur)
    float prodscal (vecteur3d &) ;        // produit scalaire
    void affiche () ;                     // affichage composantes
} ;
```

Solution

```
      /* définition de la classe vect3d */
#include <iostream.h>
vecteur3d vecteur3d::somme (vecteur3d & v)
{    vecteur3d res ;
     res.x = x + v.x ;
     res.y = y + v.y ;
     res.z = z + v.z ;
     return res ;
}
float vecteur3d::prodscal (vecteur3d & v)
{    return ( v.x * x + v.y * y + v.z * z) ;
}
void vecteur3d::affiche ()
{    cout << "< " << x << ", " << y << ", " << z << ">" ;
}
```

Solution

```
    /* programme d'essai de la classe vecteur3d */
main()
{   vecteur3d v1 (1,2,3), v2 (3,0, 2), w ;
    cout << "v1 = " ; v1.affiche () ; cout << "\n" ;
    cout << "v2 = " ; v2.affiche () ; cout << "\n" ;
    cout << "w = " ;  w.affiche () ;  cout << "\n" ;
    w = v1.somme (v2) ;
    cout << "w = " ;  w.affiche () ;  cout << "\n" ;
    cout << "V1.V2 = " << v1.prodscal (v2) << "\n" ;
}
```

Objet dynamique

- Ils sont créés par l'opérateur new, comme dans cet exemple:
  - Point * adp;
  - adp = new point (2.5, 5.32) ;
- L'accès au membres d'un objet dynamique est comme pour les variables ordinaires.
  - (*adp).affiche() ou adp->affiche()
- Il sont détruits à la demande en utilisant l'opérateur delete : delete adp.

Exercice

Comment concevoir le type classe *chose* de façon à ce que ce petit programme :

```
main()
{
    chose x ;
    cout << "bonjour\n" ;
}
```

fournisse les résultats suivants :

```
création objet de type chose
bonjour
destruction objet de type chose
```

Que fournira alors l'exécution de ce programme (utilisant le même type *chose*) :

```
main()
{
   chose * adc = new chose
}
```

Solution

Il suffit de prévoir dans le constructeur de *chose*, l'instruction :

```
cout << "création objet de type chose\n" ;
```

et dans le destructeur, l'instruction :

```
cout << "destruction objet de type chose\n" ;
```

Dans ce cas, le deuxième programme fournira simplement à l'exécution (puisqu'il crée un objet de type *chose* sans jamais le détruire) :

```
création objet de type chose
```

Exercice

Créer une classe *point* ne contenant qu'un constructeur sans arguments, un destructeur et un membre donnée privé représentant un numéro de point (le premier créé portera le numéro 1, le suivant le numéro 2, et ainsi de suite…). Le constructeur affichera le numéro du point créé et le destructeur affichera le numéro du point détruit. Ecrire un petit programme d'utilisation créant dynamiquement un tableau de 4 points et le détruisant.

Solution

```
#include <iostream.h>
class point
{   int num;
    static nb_points ;
  public :
    point ()
       {  num = ++nb_points ;
          cout << "création point numéro    : " << num << "\n" ;
       }
    ~point ()
       {  cout << "Destruction point numéro : " << num << "\n" ;
       }
} ;

main()
{  point * adcourb = new point [4] ;
   int i ;
   delete [4] adcourb ;
}
```

Constructeur de recopie

- Le but de ce type de constructeur est d'initialiser un objet lors de son instanciation à partir d'un autre objet
- Si aucun constructeur de l'une des formes type (type &) ou type (const type &) (type désignant le type de l'objet) n'a été défini, il y aura appel d'un « constructeur de recopie par défaut »; ce dernier recopie les valeurs des différents membres donnée de l'objet, comme le fait l'affectation.
- Des problèmes peuvent alors se poser dès lors que l'objet contient des pointeurs sur des parties dynamiques.

Constructeur de recopie

- Si un constructeur de la forme type (type &) ou, mieux, type (const type &) existe, il sera appelé. Notez qu'alors le constructeur de recopie par défaut n'est plus appelé. C'est donc au constructeur de recopie de prévoir la recopie de tous les membres de l'objet.
- Point a=Point (1.3, 4.5) : il n'y a pas d'appel d'un constructeur de recopie par défaut, précédé de la création d'un objet temporaire. La déclaration précédente est rigoureusement équivalente à : Point a (1.3, 4.5);

Programmation C++

- fonctions amies,
- surcharge des opérateurs,
- héritage simple,

Fonction amie

- En principe, l’encapsulation interdit à une fonction membre d’une classe ou toute fonction d’accéder à des données privées d’une autre classe.
- Mais grâce à la notion d’amitié entre fonction et classe, il est possible, lors de la définition de cette classe d’y déclarer une ou plusieurs fonctions (extérieurs de la classe) amies de cette classe.

Fonction amie

- Une fonction amie d’une classe est une fonction qui, sans être membre de la classe, a néanmoins accès à toutes les données et fonctions-membres de cette classe, qu’elles soient publiques, protégées ou privées.
- Une telle fonction amie doit être déclarée à l’intérieur de la déclaration de la classe.

Fonction amie

- Il existe plusieurs situations d’amitiés :
  - Fonction indépendante (ou libre), amie d’une classe.
  - Fonction membre d’une classe, amie d’une autre classe.
  - Fonction amie de plusieurs classes.
  - Toutes les fonctions membres d’une classe, amies d’une autre classe.

Fonction libre amie d'une classe

- Elle est déclarée dans la classe avec le mot-clé friend.
- Exemple:

```
class point{
   int x,y;
public :
   point(int a=0,int b=0) {x=a; y=b;}
   friend int coincide(point,point);};
//-------------------------------------
int coincide(point p1,point p2){
   if(p1.x==p2.x && p1.y==p2.y) return 1; else return 0;
}
```

Fonction libre amie d’une classe

```
main(){
   point o1(15,2), o2(15,2), o3(13,25);
   if(coincide(o1,o2))
         cout<<”les objets coïncident\n”;
   else
         cout<<”les objets sont différents\n”;
   if(coincide(o1,o3))
         cout<<”les objets coïncident\n”;
   else
         cout<<”les objets sont différents\n”;
}
```

Exercice

Soit la classe *point* suivante :

```
class point
{   int x, y ;
   public :
    point (int abs=0, int ord=0)
      { x = abs ; y = ord ;
      }
} ;
```

Ecrire une fonction indépendante *affiche*, amie de la classe point, permettant d'afficher les coordonnées d'un point. On fournira séparément un fichier source contenant la nouvelle déclaration (définition) de *point* et un fichier source contenant la définition de la fonction *affiche*. Ecrire un petit programme (*main*) qui crée un point de classe automatique et un point de classe dynamique et qui en affiche les coordonnées.

Fonction membre d’une classe amie d’une autre

```
class A; // informe le compilateur qu’il existe une classe nommée A
class B{
   public:
   int f(int,A);};
//-----------------------------------------------
class A{
   public :
   friend int B::f(int,A);};
//-----------------------------------------------

int B::f(int,A){ … }
```

Fonction membre d’une classe amie d’une autre

- Pour compiler correctement la déclaration de la classe ‘A’, contenant une déclaration d’une fonction amie, il faut :
  - Définir ‘B’ avant ‘A’.
  - Déclarer ‘A’ avant ‘B’.

Fonction amie de plusieurs classes

- Toute fonction membre ou indépendante, peut être amie de plusieurs classes

```
class B;
class A{
public :
   friend void f(A,B);};
//--------------------------------
class B{
public :
   friend void f(A,B);};
//----------------------------
void f(A,B){
……}
```

Toutes les fonction d'une classe sont amies d'une autre classe

- Au lieu de faire autant de déclarations de fonctions amies qu'il y a de fonctions membres, on peut résumer toutes ces déclarations en une seule.
- **Exemple** : 'friend class B;' déclarée dans la classe 'A' signifie que toutes les fonctions membres de la classe 'B' sont amies de la classe 'A'.
- **Remarque : -** Pour compiler la déclaration de la classe 'A', il suffit de la faire précéder de : 'class B ;'
  - Ce type de fonction évite d'avoir à déclarer, les entêtes des fonctions concernées par l'amitié.

Exemple

```
#include <stdio.h>
class Hote;
class Amie {
public:
    void print_hote(Hote h) {
        printf("%d\n", h.i); // Accède à la donnée privée de h.
        return ; } };
class Hote {
    friend class Amie; // Toutes les méthodes de Amie sont
    amies.
    int i; // Donnée privée de la classe Hote.
public:
    Hote(void) { i=0; return ; } };
int main(void) {
    Amie a;   a.print_hote();    return 0; }
```

Surcharge des fonctions

- Il est interdit en C de définir plusieurs fonctions qui portent le même nom.
- En C++, cette interdiction est levée.
- Le compilateur peut différencier deux fonctions en regardant le type des paramètres qu'elle reçoit.

```
float test(int i, int j) {
    return (float) i+j; }
float test(float i, float j) {
    return i*j;}
```

Exercice

- Ecrire un programme C++ qui permet, à l'aide des fonctions, de calculer la somme de deux ou trois entiers (Utiliser deux méthodes).

Exemple d'exécution :

Donner trois entiers: 3 9 8

Methode 1:

3 + 9 = 12

3 + 9 + 8 = 20

Methode 2:

3 + 9 = 12

3 + 9 + 8 = 20

Solution

```
#include<iostream>
using namespace std;
//Methode 1 : Avec paramètre par défaut
int somme1(int x, int y, int z=0){
    return x+y+z ;}
//Methode 2 : Avec surcharge des fonctions
int somme2(int x, int y, int z){
   return x+y+z;}
int somme2(int x, int y){
    return x+y;}
```

Solution

```
int main(){
   int a, b, c;
   cout << "Donner trois entiers: ";
   cin >> a >> b >> c;
   cout << "Methode 1: " << endl;
   cout << a << " + " << b << " = " << somme1(a,b) << endl;
   cout << a << " + " << b << " + " << c << " = " <<
   somme1(a,b,c) << endl;
   cout << "Methode 2: " << endl;
   cout << a << " + " << b << " = " << somme2(a,b) << endl;
   cout << a << " + " << b << " + " << c << " = " <<
   somme2(a,b,c) << endl;
return 0;}
```

Surcharge des opérateurs

- En C++, on peut *surcharger la plupart des opérateurs usuels du langage, c'est-à-dire les reprogrammer* pour que, dans un certain contexte, ils fassent autre chose que ce qu'ils font d'habitude.

Surcharge des opérateurs

Parmi les nombreux exemples que l'on pourrait citer :

- myString + yourString pourrait servir à concaténer deux objets string
- maDate++ pourrait servir à incrémenter un objet Date
- a * b pourrait servir à multiplier deux objets Nombre

Surcharge des opérateurs

- La surcharge d'un opérateur <*op*> *se fait en déclarant, au sein d'une classe, une fonction membre* appelée **operator** <*op*>.
- *Plusieurs cas peuvent se présenter, selon que <op> est un opérateur* ***unaire*** *(c'est-à-dire à un argument) ou* ***binaire*** *(c'est-a-dire à deux arguments).*

Surcharge des opérateurs : unaire

- surcharger l'opérateur unaire « - » pour qu'il calcule l'opposé d'un nombre complexe
- Dans la classe **Complexe**, nous déclarons la fonction membre publique suivante :

```
Complexe operator-();
```

- que nous définissons ensuite en utilisant le constructeur paramétré de la classe :

```
Complexe Complexe::operator-(){
   return Complexe(-re, -im);}
```

Surcharge des opérateurs: unaire

- Par la suite, si z est une variable de type Complexe, on pourra écrire tout simplement l’expression –z pour désigner l’opposé de z ou z.operator-()

Surcharge des opérateurs: binaire

- surcharger l'opérateur binaire « - » pour qu'il calcule la différence de deux nombres complexes.
- Dans la classe **Complexe**, nous déclarons la fonction membre publique suivante :

```
Complexe operator-(Complexe u);
```

- que nous définissons ensuite en utilisant également le constructeur paramétré de la classe :

```
Complexe Complexe::operator-(Complexe u){
  return Complexe(re - u.re, im - u.im);}
```

Surcharge des opérateurs: binaire

- Par la suite, si z1 et z2 sont deux variables de type Complexe, on pourra écrire tout simplement l'expression z1 - z2 pour désigner le nombre complexe obtenu en soustrayant z2 de z1, ou à l'aide de l'expression

  z1.operator-(z2)

Surcharge des opérateurs de comparaison

```
bool Nombre::operator==(Nombre &n){
   return (valeur == n→valeur);}
bool Nombre ::operator<(Nombre &n){
   return (valeur < n→valeur);}
bool Nombre ::operator>(Nombre &n){
   return (valeur > n→valeur);}
```

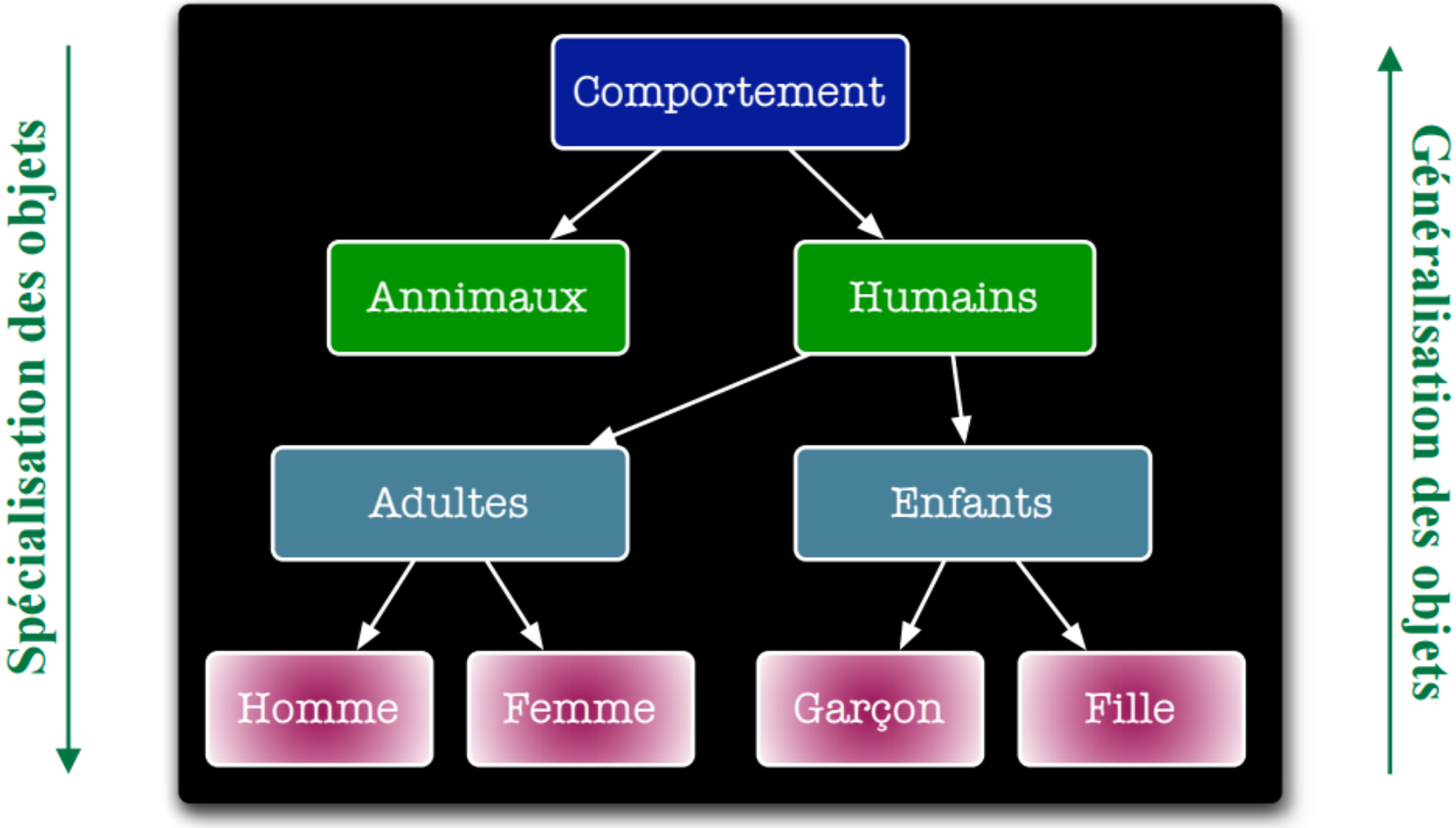
- Nombre a(2), b(3);
- bool c = (a == b);
- bool d = (a > b);
- bool e = (d < c);

Héritage en C++

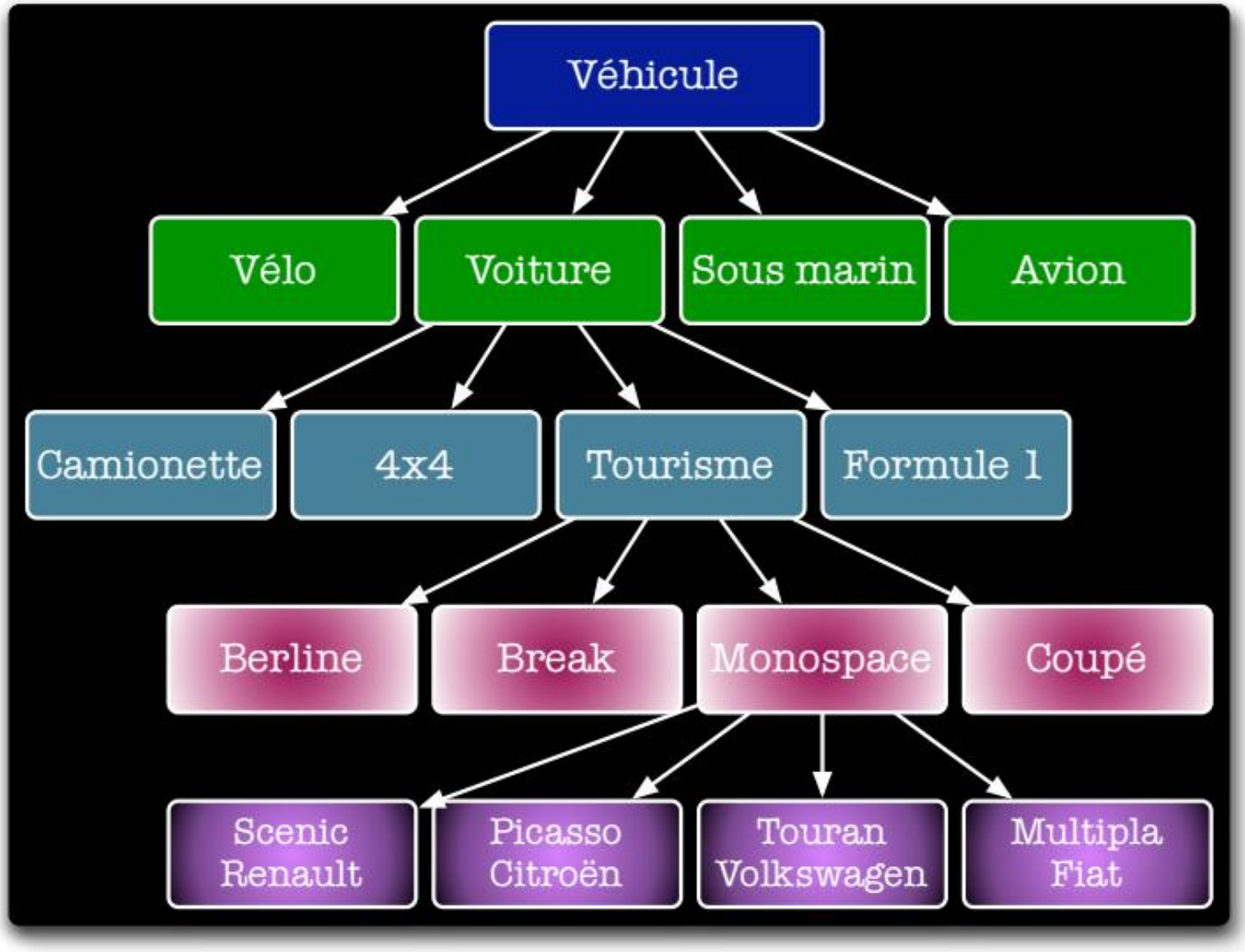
Définition

- L’héritage (ou spécialisation, ou dérivation) permet d’ajouter des propriétés à une classe existante pour en obtenir une nouvelle plus précise.
- L’idée est : "un B est un A avec des choses en plus".
- Exemple : Un étudiant est une personne, et a donc un nom (et un prénom, ...). De plus, il a un numéro INE.

Exemple


Spécialisation des objets
Comportement
Annimaux
Humains
Adultes
Enfants
Homme
Femme
Garçon
Fille
Généralisation des objets

Exemple


Véhicule
Vélo
Voiture
Sous marin
Avion
Camionette
4x4
Tourisme
Formule 1
Berline
Break
Monospace
Coupé
Scenic Renault
Picasso Citroën
Touran Volkswagen
Multipla Fiat

Exemple

```
// Personne est la classe parente ou mère de Etudiant
   (en anglais superclass)
class Personne {
private: string nom;
public: string getNom() { return nom; } };
// Etudiant est la classe fille ou dérivée de Personne :
   Etudiant hérite ou descend de Personne
class Etudiant : public Personne {
private: string ine;
public: string getINE() { return ine; } };
```

Exemple

Personne unePersonne; Etudiant unEtudiant;

unePersonne.getNom(); // légal : unePersonne est une Personne

unEtudiant.getINE(); // légal : unEtudiant est un Etudiant

unEtudiant.getNom(); // légal : unEtudiant est une Personne (par héritage)

unePersonne.getINE(); // illégal : unePersonne n'est pas un Etudiant (erreur: ' class Personne' has no member named 'getINE')

Conversion automatique

- Si B hérite de A, alors toutes les instances de B sont aussi des instances de A, et il est donc possible de faire :

Exemple

```
Personne unePersonne; Etudiant unEtudiant;
unePersonne = unEtudiant; // Ok !
// Propriété conservée lorsqu'on utilise des pointeurs
    Personne *pUnePersonne;
    Etudiant *pUnEtudiant = &unEtudiant;
// pointer sur un B c'est avant tout pointer sur un A
pUnePersonne = pUnEtudiant;
// Évidement, l'inverse n'est pas vrai :
    unEtudiant = unePersonne; // ERREUR !
// Pareil pour les pointeurs :
    pUnePersonne = &unePersonne;
    pUnEtudiant = pUnePersonne;
    // ERREUR ! // pointer sur un A n'est pas pointer sur un B
```

Conversion automatique

- Conclusion : Traiter un type dérivé comme s'il était son type de base est appelé transtypage ascendant ou surtypage (upcasting).

- A l'opposé, le transtypage descendant (downcast) pose un problème particulier car leur vérification n'est possible qu'à l'exécution. Il nécessite l'utilisation d'opérateur de cast : dynamic_cast.

Principes

- L’héritage permet :
  - la réutilisation du code déjà écrit
  - l’ajout de nouvelles fonctionnalités
  - la modification d’un comportement existant (redéfinition)

Principes

```
// Classe de base
class Personne {
    private: string nom;
    public: string getNom() const;
    void afficher()  { cout << nom << endl; } };
// Classe dérivée
class Etudiant : public Personne {
    private: string ine;
    public: string getINE() ; // ajout de nouvelles
    fonctionnalités
// modification d'un comportement existant
    void afficher() {
        Personne::afficher(); // réutilisation du code déjà écrit
        cout << ine << endl; } };
```

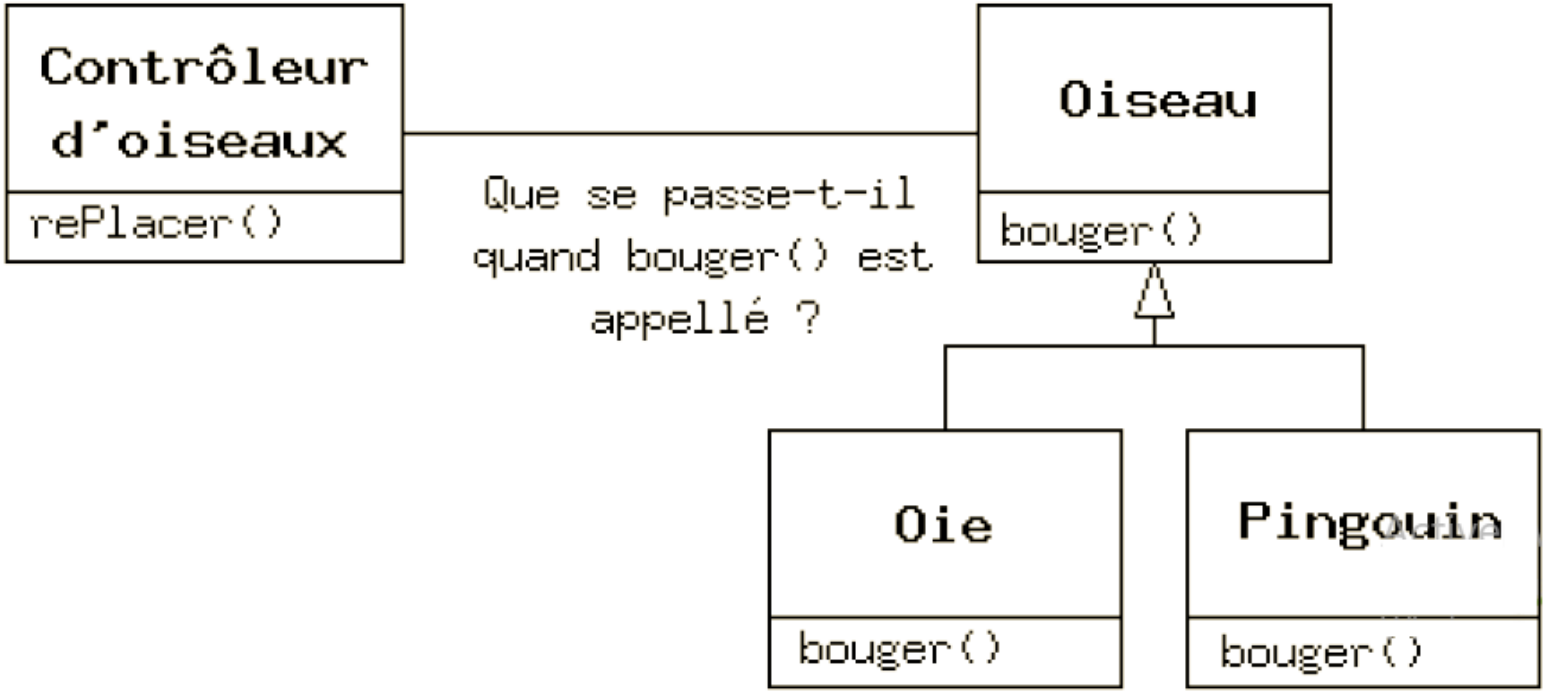
Autre exemple

```
class Point {
   public: int x; int y;
   Point(int a, int b) { x = a; y = b; };
   ~Point();
   void deplace(int dx, int dy) { x += dx; y += dy; };
   void affiche() { cout << "x = " << x << "y = " << y << endl;
   };
};
class PointCol : public Point {
   public: int couleur;
   PointCol(int, int, int);
   ~Point();
   void colore(int c) { couleur = c }; };
```

- Dans cet exemple, on a défini une classe "dérivée" (PointCol) à partir d'une classe "de base" (Point).
- On dit que la classe PointCol « hérite » de la classe Point.
- Cela signifie que les membres de la classe Point sont des membres de la classe PointCol.

Comportement Polymorphe (1/4)

- Par exemple, dans le diagramme suivant, l'objet Contrôleur d'oiseaux travaille seulement avec des objets Oiseaux génériques, et ne sait pas de quel type ils sont. C'est pratique du point de vue de Contrôleur d'oiseaux, car il n'a pas besoin d'écrire du code spécifique pour déterminer le type exact d'Oiseau avec lequel il travaille, ou le comportement de cet Oiseau.



Comportement Polymorphe (2/4)

- Comment se fait-il donc que, lorsque bouger() est appelé tout en ignorant le type spécifique de l'Oiseau, on obtienne le bon comportement (une Oie court, vole ou nage, et un Pingouin court ou nage) ?
- La réponse constitue l'astuce fondamentale de la programmation orientée objet : le compilateur ne peut faire un appel de fonction au sens traditionnel du terme. Un appel de fonction généré par un compilateur non orienté objet crée ce qu'on appelle une association prédéfinie : le compilateur génère un appel à un nom de fonction spécifique, et l'éditeur de liens résout cet appel à l'adresse absolue du code à exécuter.
- En POO, le programme ne peut déterminer l'adresse du code avant la phase d'exécution, un autre mécanisme est donc nécessaire quand un message est envoyé à un objet générique.

Comportement Polymorphe (3/4)

- Pour résoudre ce problème, les langages orientés objet utilisent le concept d'**association tardive. Quand un objet reçoit un message, le** code appelé n'est pas déterminé avant l'exécution. Le compilateur s'assure que la fonction existe et vérifie le type des arguments et de la valeur de retour, mais il ne sait pas exactement quel est le code à exécuter.
- Pour créer une association tardive, le compilateur C++ insère une portion spéciale de code en lieu et place de l'appel absolu. Ce code calcule l'adresse du corps de la fonction, en utilisant des informations stockées dans l'objet. Ainsi, chaque objet peut se comporter différemment suivant le contenu de cette portion spéciale de code. Quand un objet reçoit un message, l'objet sait quoi faire de ce message.

Comportement Polymorphe (4/4)

- On déclare qu'on veut une fonction qui ait la flexibilité des propriétés de l'association tardive en utilisant le mot-clé virtual.
- On n'a pas besoin de comprendre les mécanismes de virtual pour l'utiliser, mais sans lui on ne peut pas faire de la programmation orientée objet en C++.
- En C++, on doit se souvenir d'ajouter le mot-clé virtual (devant une méthode) parce que, par défaut, les fonctions membres ne sont pas liées dynamiquement. Les fonctions virtuelles permettent d'exprimer des différences de comportement entre des classes de la même famille.
- Ces différences sont ce qui engendre un **comportement polymorphe**

Exemple : comportement non polymorphe (1/2)

```
#include <iostream>
using namespace std;
class Forme {
public:
    Forme() { cout << "constructeur Forme <|- "; }
    void dessiner() { cout << "je dessine ... une forme ?\n"; }};
class Cercle : public Forme {
public:
    Cercle() { cout << "Cercle\n"; }
    void dessiner() { cout << "je dessine un Cercle !\n"; }};
class Triangle : public Forme {
public:
    Triangle() { cout << "Triangle\n"; }
    void dessiner() { cout << "je dessine un Triangle !\n"; }};
```

Exemple : comportement non polymorphe (2/2)

```
void faireQuelqueChose(Forme &f){
   f.dessiner(); // dessine une Forme
}
int main(){
   Cercle c;
   Triangle t;
   faireQuelqueChose(c); // avec un cercle
   faireQuelqueChose(t); // avec un triangle
   return 0;
}
```

- L’exécution du programme d’essai nous montre que nous n’obtenons pas un comportement polymorphe puisque c’est la méthode dessiner() de la classe Forme qui est appelée :
- constructeur Forme <|- Cercle
- constructeur Forme <|- Triangle
- je dessine ... une forme ?
- je dessine ... une forme ?

Exemple : comportement polymorphe (1/2)

```
#include <iostream>
using namespace std;
class Forme {
public:
    Forme() { cout << "constructeur Forme <- "; }
    // la méthode dessiner sera virtuelle et fournira un comportement polymorphe
    virtual void dessiner() { cout << "je dessine ... une forme ?\n"; }
};
class Cercle : public Forme {
public:
    Cercle() { cout << "Cercle\n"; }
    void dessiner() { cout << "je dessine un Cercle !\n"; }
};
class Triangle : public Forme {
public:
    Triangle() { cout << "Triangle\n"; }
    void dessiner() { cout << "je dessine un Triangle !\n"; }
};
```

Exemple : comportement polymorphe (2/2)

```
void faireQuelqueChose(Forme &f){
   f.dessiner(); // dessine une Forme
}
int main(){
   Cercle c;
   Triangle t;
   faireQuelqueChose(c); // avec un cercle
   faireQuelqueChose(t); // avec un triangle
   return 0;
}
```

- L’exécution du programme d’essai nous montre maintenant que nous obtenons un comportement polymorphe puisque c’est la “bonne“ méthode dessiner() qui est appelée :
- constructeur Forme <|- Cercle
- constructeur Forme <|- Triangle
- je dessine un Cercle !
- je dessine un Triangle !

Redéfinition de fonctions membre et de membres

- Une fonction membre de la classe dérivée peut avoir la même signature qu'une fonction de la classe de base.
- On dit alors que la fonction membre de la classe de base est « Redéfinie » dans la classe dérivée.
- Quand on est dans la classe dérivée, on appelle la fonction de la classe dérivée de façon habituelle.
- Si on veut appeler la fonction membre de la classe de base, il faut préciser le nom de la classe de base :: en préfixe. Exemple:

```
class PointCol : public Point { ...
    void affiche() { Point::affiche(); cout << "couleur " <<
    couleur << endl; }; };
main() {
    PointCol p(10, 20, 5);
    p.affiche();
    p.Point::affiche();
    p.deplace(2,4);
    p.affiche();
    p.colore(2);
    p.affiche(); }
```

- Pour les membres, c’est analogue. Exemple:
  - class A { ... int a; char b; ... };
  - class B: public A {float a; ... };
- Si b est de type B, b.a fait référence au membre a de type float de la classe B.
- On accède au membre **a** de type int par b.A::a
- Le membre **a** de la classe B s'ajoute à celui de la classe A. Il ne le remplace pas.
- NB: on dit REdéfinition et pas SURdéfinition. (Surdéfinition signifie changement de signature de la fonction au sein d’une même classe).

Appel des constructeurs et destructeurs

- En fait les constructeurs et destructeurs obéissent à des règles précises.
- On peut transmettre les paramètres du constructeur dérivé au constructeur de base:

```
class PointCol : public Point {
    int couleur;
public:
    PointCol(int x, int y, int c) : Point(x,y) { couleur = c };
    ~Point(); ... };
```

- On peut toujours mettre des arguments par défaut.

- Lors de l'appel du constructeur de la classe dérivée, il se produit dans l'ordre:
  - appel du constructeur de base,
  - exécution du corps du constructeur dérivé.
- Lors de l'appel du destructeur de la classe dérivée, il se produit dans l'ordre:
  - exécution du corps du destructeur dérivé,
  - appel du destructeur de base.

```
class Point { ...
    Point (int abs=0, int ord=0) { x=abs; y = ord; cout << "++ construc
    Point " << x << " " << y << endl; };
    ~Point () { cout << "-- destruc Point " << x << " " << y " endl; };
    ... }
class PointCol : public Point { ...
    PointCol (int abs=0, int ord=0, int c=1) : Point(abs, ord) {
    couleur = c; cout << "++ construc PointCol " << couleur << endl; };
    ~PointCol () { cout << "-- destruc PointCol " << couleur << endl; };
    ... }
main () {
    PointCol a(10, 15, 3);
    PointCol b(2, 3); PointCol c(12); PointCol d;
    pointCol * p; p = new PointCol(12, 25); delete p;
}
```

la sortie est :

```
++ construc Point 10 15
++ construc PointCol 3
++ construc Point 2 3
++ construc PointCol 1
++ construc Point 12 0
++ construc PointCol 1
++ construc Point 0 0
++ construc PointCol 1
++ construc Point 12 25
++ construc PointCol 1
-- destruc PointCol 1
-- destruc Point 12 25
-- destruc PointCol 1
-- destruc Point 0 0
-- destruc PointCol 1
-- destruc Point 12 0
-- destruc PointCol 1
-- destruc Point 2 3
-- destruc PointCol 3
-- destruc Point 10 15
```

Constructeur par recopie.

- Si la classe dérivée n'a pas de constructeur par recopie, alors appel du constructeur par défaut (copie des attributs un à un).
- Si la classe dérivée a un constructeur par recopie B(B&b), il n'y a pas forcément d'appel automatique au constructeur par recopie de la classe de base: on peut le spécifier par B (B & b) : A (b)
- conversion implicite de b en un objet de la classe A

Contrôle des accès

- Avec le mot-clé ‘**protected**’ implique que:
- La classe dérivée a accès aux membres protégés de la classe de base.
- Les classes utilisatrices de la classe dérivée non. Exemple :

```
class Point {
   int x; int y;
public: Point(int a, int b) { x = a; y = b; };
   ~Point();
protected:
   void deplace(int dx, int dy) { x += dx; y += dy; };
   void affiche() { cout << "x = " << x << "y = " << y << endl; }; };
```

Intérêt du statut protégé:

- Les membres protégés sont comparables à des membres privés pour un utilisateur de la classe et à des membres publiques pour la classe dérivée.

- Héritage public :
  - Il donne aux membres publics et protégés de la classe de base le même statut dans la classe dérivée.
- Héritage privé :
  - Il donne aux membres publics et protégés de la classe de base le statut de membres privés dans la classe dérivée.

Compatibilité entre objets d'une classe de base et objets d'une classe dérivée

- Puisque la relation d’héritage est une relation forte, on peut se demander si l’on peut affecter un objet de la classe de base dans un objet de la classe dérivée ou pas, et inversement.
- L’idée est que tout objet de la classe dérivée est un objet de la classe de base et que C++ fait des conversions implicites de type.

- On peut mettre un objet de type dérivé vers un type de base mais pas l’inverse.
- Idem : on peut mettre un pointeur sur objet de type dérivé vers un pointeur sur objet de type de base et pas l’inverse.
- Si B est une classe dérivée de la classe A et si on a :
  - A a; B b;
- alors a = b ; est légal alors que b = a ; est illégal.
- Pour s’en convaincre, imaginons que la classe C dérive de la classe A et que l’on ait : C c; Puisque c est dans la classe C, il est dans la classe A et donc si on pouvait faire b=a; on pourrait faire b=c; ce qui est absurde.
- Donc on ne peut pas faire b=a;

- Si on a : A * pa; B * pb; alors pa = pb ; est légal alors que pb = pa ; est illégal.
- Par contre, très souvent, dans une classe dérivée, le programmeur sait que le pointeur pa est de la classe dérivée donc il peut violer la règle avec l'opérateur de cast en écrivant :
  - pb = (B*) pa ;
- Evidemment, cette opération peut amener des catastrophes si l'objet n'est pas de la classe dérivée B mais d'une autre classe dérivée C, complètement différente de la classe B.

- Le typage des objets est statique en C++. Cela signifie que le type d'un objet est connu à la compilation et ne change plus ensuite.
- Point p(3, 5); PointCol pc(8,6,2); Point * pa = &p; PointCol * pb = &pc;
- pa->affiche() appellera Point::affiche()
- pb->affiche() appellera PointCol::affiche()
- Mais si on fait : pa = pb;
- malheureusement pa->affiche() appelle Point::affiche().
- Le choix de la méthode à appeler est fait statiquement (c'est-à-dire par le compilateur).
- Par exemple, pa->colore(4); est une instruction illégale.

Exercice

On dispose d'un fichier nommé *point.h* contenant la déclaration suivante de la classe *point* :

```
class point
{   float x, y ;
  public :
    void initialise (float abs=0.0, float ord=0.0)
       { x = abs ; y = ord ;
       }
    void affiche ()
       { cout << "Point de coordonnées : " << x << " " << y << "\n" ;
       }
    float abs ()  { return x ; }
    float ord ()  { return y ; }
} ;
```

Exercice

a) Créer une classe *pointa*, dérivée de *point* comportant simplement une nouvelle fonction membre nommée *rho*, fournissant la valeur du rayon vecteur (première coordonnée polaire) d'un point.

b) Même question, en supposant que les membres x et y ont été déclarés protégés (*protected*) dans *point*, et non plus privés.

c) Introduire un constructeur dans la classe *pointb*.

d) Quelles sont les fonctions membre utilisables pour un objet de type *pointb* ?

Exercice

On dispose d'un fichier *point.h* contenant la déclaration suivante de la classe *point* :

```
#include <iostream.h>
class point
{   float x, y ;
  public :
    point (float abs=0.0, float ord=0.0)
       { x = abs ; y = ord ;
       }
    void affiche ()
       { cout << "Coordonnées : " << x << " " << y << "\n" ;
       }

    void deplace (float dx, float dy)
       { x =  x + dx ;  y = y + dy ;
       }
} ;
```

a) Créer une classe *pointcol*, dérivée de *point*, comportant :

- un membre donnée supplémentaire *cl*, de type *int*, destiné à contenir la "couleur" d'un point
- les fonctions membre suivantes :

  * *affiche* (redéfinie), qui affiche les coordonnées et la couleur d'un objet de type *pointcol*,

  * *colore* (*int couleur*), qui permet de définir la couleur d'un objet de type *pointcol*,

  * un constructeur permettant de définir la couleur et les coordonnées (on ne le définira pas "en ligne".

b) Que fera alors précisément cette instruction :

```
pointcol (2.5, 3.25, 5) ;
```

Programmation C++

- classe abstraite,

Classe Abstraite

- Soit **ObjetGeometrique**, qui possède deux méthodes **surface()** et **perimetre()**, une super-classe des classes **Triagle** et **Ellipse**.

- L'implémentation des méthodes surface() ou perimetre() dépend de la forme géométrique qui hérite de la classe ObjetGeometrique.

Classe Abstraite

- Les classes Triangle et Ellipse doivent remplacer ces méthodes par une implémentation significative.

- Alors que nous ne devons pas écrire une implémentation de ces méthodes dans la classe ObjetGeometrique.

- Ces méthodes serons déclarées comme abstraites (abstract) dans la classe ObjetGeometrique.

Classe Abstraite

- Pensez à des méthodes abstraites comme «placeholders» pour les méthodes qui seront éventuellement définis dans certaines sous-classes de la classe actuelle.

- En faisant cela, nous reconnaissons que ObjetGeometrique n’est pas complète dans un certain sens

- Il serait inapproprié d’instancier un objet de cette classe.

Classe Abstraite

- Plus généralement, les classes avec des méthodes abstraites doivent se déclarer abstraite.

- Autrement dit, une méthode abstraite ne peut être contenue dans une classe non abstraite.

- De même, si une sous-classe d'une super-classe abstraite n’implémente pas toutes les méthodes abstraites de la super-classe, elle doit être déclarée comme abstraite.

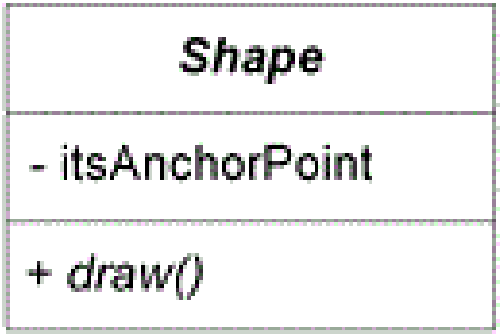
Classe Abstraite

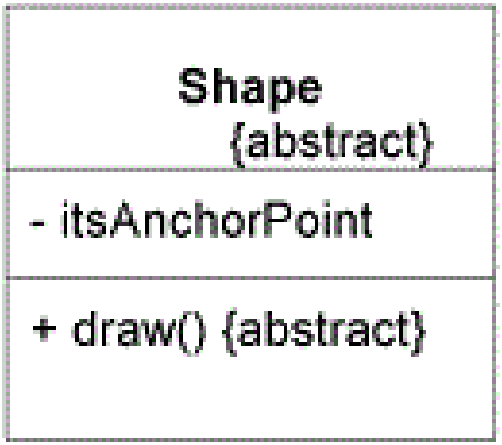
- Donc, si on désire étendre une classe abstraite, toutes les méthodes de la superclasse doivent être implémentées, même si elles ne sont pas utilisées dans la sous-classe.

- Une sous-classe peut être abstraite, même si sa superclasse est complète. Par exemple, la classe Objet est complète, mais peut avoir une sous-classe abstraite comme ObjetGeometrique.

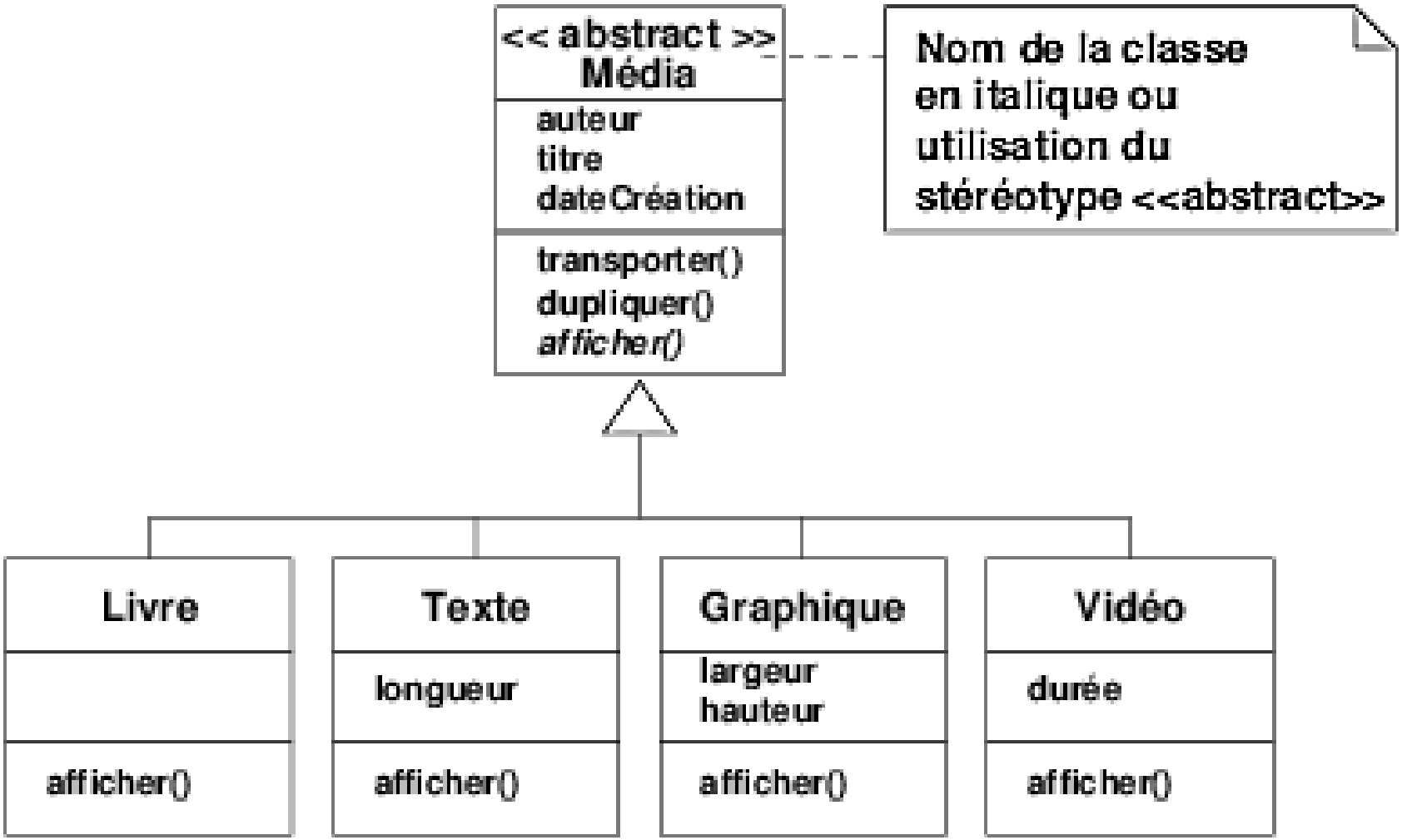
Classe Abstraite : Notation graphique

- Il y a deux façons d'indiquer qu'une classe ou une méthode est abstraite:
  - Vous pouvez écrire le nom en italique,
  - ou vous pouvez utiliser la propriété {abstract} ou <<abstract >>

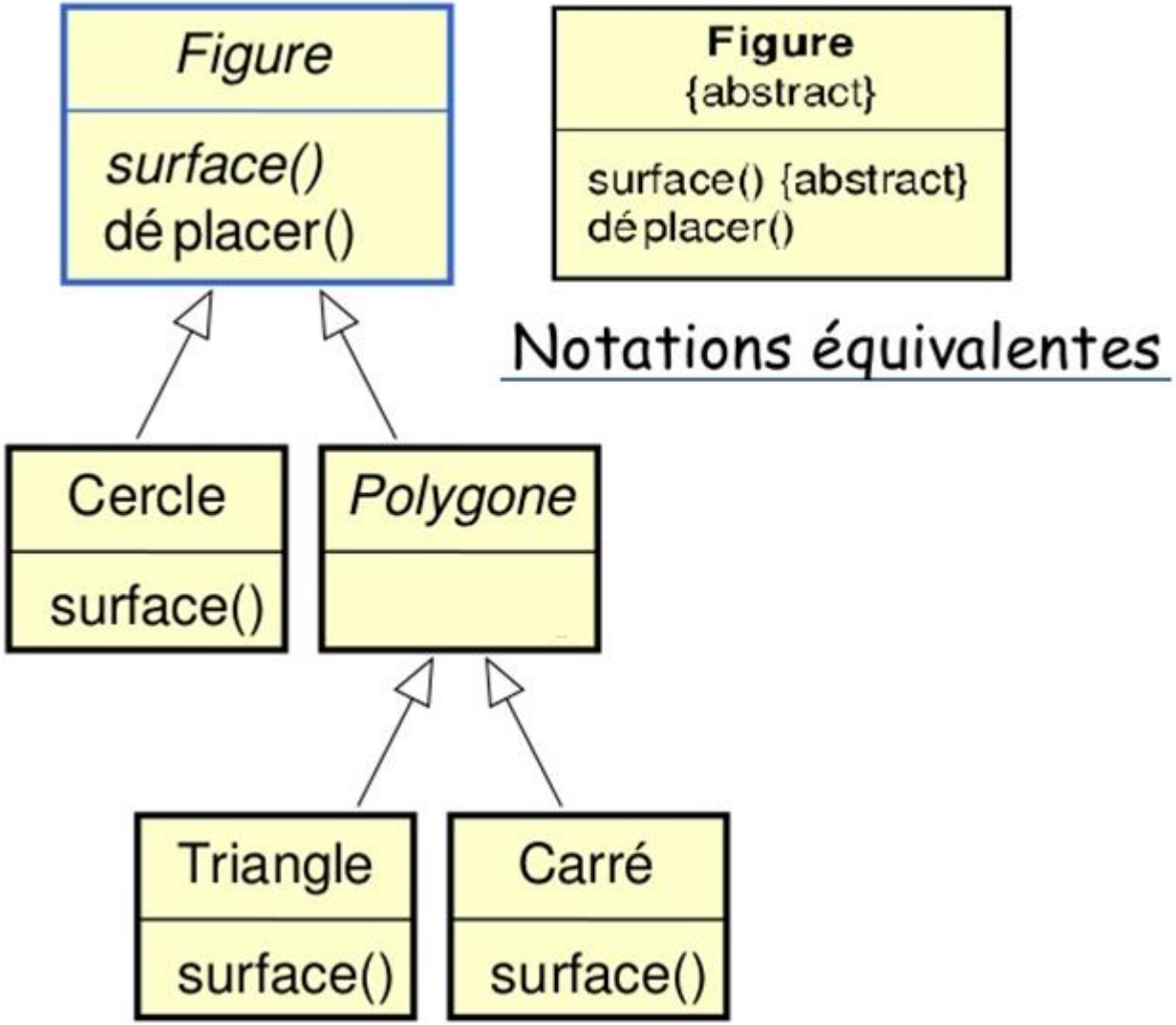




Classe Abstraite : exemple



Classe Abstraite : exemple



Classe Abstraite : exemple

- Dans certains cas, nous pouvons désirer implémenter deux objets proches possédant une partie de méthodes et de propriétés en commun comme *Voiture* et *Moto*.
- Certaines de ces méthodes comme *rouler()* sont totalement identiques d'un objet à l'autre alors que certaines autres diffèrent comme la méthode *seGarer()* qui est différente pour une *Voiture* et pour une *Moto*.

- Pour ce faire, l'OO introduit la notion de classe abstraite.

Classe Abstraite : règles et définitions

- Une méthode est dite abstraite lorsqu'on connaît son entête, mais pas la manière dont elle peut être réalisée.

- Une classe est dite abstraite lorsqu'elle définit au moins une méthode abstraite ou lorsqu'une classe parent contient une méthode abstraite non encore réalisée.

- On ne peut instancier une classe abstraite : elle est vouée à se spécialiser.

Classe Abstraite : règles et définitions

- Une classe abstraite peut très bien contenir des méthodes concrètes.
- Spécialisation nécessaire jusqu'à obtenir des classes instanciables
- Impose un comportement commun à toutes les classes dérivées
- Une classe abstraite pure ne comporte que des méthodes abstraites.
- Une telle classe est appelée une interface.